QuickFlow 構成図

中学校数学の小テストを「配信 → 手書き → 提出 → 採点 → 成績出力」までつなぐ

通常利用の主要構成

1 利用者のブラウザ

教師・管理者

教師：PDF配信・判定修正・Excel

管理者：自校の教師・生徒を管理

生徒：開く → 書く → 提出

PDF.js：教材PDFを表示

Canvas：筆跡を描く

pdf-lib：教材＋筆跡を答案PDFへ

JavaScript / Stimulus

画面操作・自動保存・状態の確認

画面と結果が返る

RailsのViewがHTMLを作る

画面から保存や結果取得を依頼

採点中は状態を定期的に確認

HTTPS

2 サーバー側：AWS EC2 / Docker

Caddy：HTTPS通信の入口

ブラウザからの通信をRailsのWeb処理へ渡す

Web処理：Puma / Rails

routes：URLと処理を結ぶ

Controller：受付・権限確認

Model：データ操作・ルール

View：表示する画面を作る

提出・保存・結果表示・Excel

登録

バックグラウンド処理

Active Job：仕事を登録

Solid Queue：待ち行列・実行

Job：処理の順番・状態を管理

Service：読取・採点などの手順

JobからServiceを呼び出す

Web・Jobの両方が保存／読込

3 データの保存先（Web・Jobで共用）

PostgreSQL

学校・利用者・課題・下書き・採点結果

業務DB / Queue / Cache / Cable

ファイル保存領域

教材PDF・提出した答案PDF

Active Storage（Disk）で管理

API通信

4 OpenAI API（外部）

教材から正答・採点条件を生成

答案PDFから手書き解答を読む

解答と正答を照合し、判定を支援

AiClientが通信の窓口

閲覧専用ゲストは別環境

学校用と別のDB・保存領域を使用

架空データの画面を閲覧する

手書き・提出・実AI処理は行わない

処理の中身：Jobが進行を管理し、Serviceが具体的な仕事を担当する

教材配信：正答を準備する

QuestionGenerationJob

→ QuestionGenerator（Service）

教材PDFから問題・正答・採点条件を取り出し、
QuestionとしてDBへ保存する。

答案提出：読む → 採点する

AnswerReadingJob → AnswerReader

AnswerGradingJob → AnswerGrader

読取結果はSubmission.extracted_textへ。
採点結果はGradingResultへ保存する。

保存と最終判定のポイント

下書き：AnswerDraftに筆跡データを保存

正式提出：Submission＋答案PDFを保存

教師判定があれば優先して集計・Excel出力。
なければAI判定を使用し、要確認は得点外。

確認日：2026-11-07｜GitHub main 17e6b186 のコードと運用記録を基に作成。本番の稼働状態・配置は今回未確認。

根拠：README / config/routes.rb・database.yml・storage.yml / app/controllers・jobs・services・javascript / 運用runbook